

Guardian — Full Build Plan

A free, open, non-technical-user-friendly Android app combining ad/tracker blocking, per-app firewall, IPS/DDoS protection, and a purpose-built local AI explanation layer — all on one shared traffic pipeline. No timeline pressure; built to be right, not fast.

Core principle

Every phase below plugs into the **same VpnService interception point**. There is one traffic pipeline and one settings model — blocklists, firewall rules, and IPS/DDoS anomaly detection are layers checked against the same connection, not separate systems. This is true for the code as much as for the user experience: one app, not three products stitched together.

Phase 1 — Core blocker (foundation for everything else)

Goal: system-wide ad/tracker/malware blocking, proven to work, before anything else is layered on.

Steps:

1. Set up the Android project (`VpnService` - based local filter, no real VPN tunnel — a local sinkhole).
2. Extract and parse the UT1 Toulouse Capitole archive (`blacklists.tar.gz`) — pull only the relevant categories: `ads` , `phishing` , `cryptojacking` , `stalkerware` , `marketingware` .
3. Pull in StevenBlack's hosts list, EasyList, and EasyPrivacy.
4. Merge all sources, normalize (strip protocol/paths, lowercase), and deduplicate.
5. Compile the merged list into a compact **Bloom filter** (target: single-digit-to-low-tens of MB, near-instant lookups) as a build-time step — the phone never parses raw text.
6. Wire the compiled filter into the `VpnService`: every outbound connection request is checked against it; matches are dropped.
7. Basic on/off toggle UI — no configuration screen yet, just enable/disable.

8. Test against real usage (compare block counts to the DDG ATP baseline already being tracked) to validate effectiveness before moving on.

Exit criteria: app runs system-wide, blocks at a rate comparable to or better than DDG App Tracking Protection's current 7-day totals, with no noticeable battery/performance hit.

Phase 2 — Per-app firewall + stats dashboard

Goal: give users control per app, and visibility into what's happening, without adding complexity to the default experience.

Steps:

1. Extend the VpnService pipeline with an allow/deny rule layer per installed app (reuses the same interception point from Phase 1 — not a separate engine).
2. Build the per-app stats view: tracking attempts blocked, apps contacted, companies

involved (mirrors the DDG ATP dashboard style already familiar).

3. Add data usage breakdown per app (separate from block count — surfaces apps burning background traffic even when nothing is blocked, e.g. the unused-but-active Disney+ case already observed).
4. Add CSV/JSON export of block logs, removing the need to manually track weekly totals.
5. Keep all of this **hidden behind one extra tap** — defaults stay a single on/off switch; per-app controls are opt-in depth, not the front screen.

Exit criteria: a non-technical user never needs this screen to be protected, but a curious user can drill in and get real per-app answers.

Phase 3 — Plain-language explanation layer (rules-based first)

Goal: turn raw block logs into insight, without committing to a trained model before it's justified.

Steps:

1. Build a **template-driven explanation engine** first — pattern rules like *"[App] has contacted [N] companies in the last hour — this is unusually high"* — fast to build, no training data needed, no size/battery cost.
2. Add company/category lookups (open trackerdb-style data) so blocks show *who* and *why*, not just a domain string — the "who and why" transparency layer.
3. Validate this is genuinely useful in daily use before considering Phase 3b.
4. **Phase 3b (later, optional)**: if the rules-based engine hits real limits, begin designing a small, purpose-built AI model — trained from scratch, narrow in scope (only reasons about tracking/blocking data), no dependency on Anora or any general-purpose assistant. Kept small deliberately so its behavior stays auditable — this is part of the zero-telemetry, provably-free story, not a bolt-on chatbot.

Exit criteria: every blocked event a user sees has a plain-language reason attached — no DNS/SNI/IPS jargon anywhere in the UI.

Phase 4 — Inference-risk warning (the genuinely novel feature)

Goal: warn users when their own protection setup has become a fingerprint — something no existing blocker does.

Steps:

1. Model the "anonymity set" concept: track which blocking config, DNS setup, and device signal combination the user is running.
2. Compare against known common configurations (popular VPNs, mainstream blocklists, default Android fingerprints) to estimate how rare the user's specific combination is.
3. Surface a plain-language warning when a setup becomes unusually unique — e.g., a self-modified blocklist or an uncommon combination of tools — rather than only ever reporting "blocked more = better."
4. This phase depends on real usage data from Phases 1–3, so it's built once the core is stable and generating enough signal to be meaningful.

Exit criteria: the app can tell a user not just "we blocked X" but "your protection itself may be making you more identifiable" — and explain what to do about it.

Phase 5 — IPS / DDoS / intrusion protection

Goal: add inbound protection (attacks aimed *at* the device) alongside the outbound protection (blocklists) built in Phases 1–3. Saved for last — hardest, most infrastructure-heavy, and highest risk of false positives if rushed.

Steps:

1. Connection-rate anomaly detection — flag/throttle abnormal bursts of inbound connection attempts (mobile equivalent of DDoS mitigation).
2. Port scan detection — recognize sequential/rapid probing patterns and block the source.
3. Lightweight signature-based intrusion detection — mobile-scale pattern matching

against known exploit/attack signatures
(conceptually a trimmed-down Snort/Suricata
idea, not the full engine).

4. Public WiFi auto-hardening — automatically
raise IPS sensitivity on untrusted/open
networks, relax it on trusted home WiFi.
5. Alert log for intrusion attempts, following the
same "show plain-language impact, not
mechanics" rule as everywhere else in the
app.
6. Reuse [[realguard]]'s existing design as the
blueprint rather than starting from zero.

Exit criteria: the app provides both halves of what
a router-based OPNsense setup would split
across separate firewall and IDS/IPS packages —
combined into the same single app and pipeline.

Ongoing, cross-phase requirements (apply to every phase)

- **Non-technical-user friendly by default.** Single
on/off switch, zero required configuration, no

jargon in the UI, advanced settings always one tap deeper — never a setup wizard.

- **One shared pipeline.** No phase introduces a separate service/engine — everything is a rule layer on the same VpnService interception point.
- **Free, always.** No premium tier, no paywalled features, no ads to fund it, no data sale. Distribution via sideload/F-Droid to avoid Play Store policy restrictions on VPN-based ad blockers.
- **Provable zero telemetry.** The app's own network traffic should be auditable/open source, so "nothing leaves the device" is checkable, not just claimed.
- **No rush.** One-year build horizon, phases can overlap where useful, but each phase should reach a real, testable exit criteria before being considered "done."